

Proyecto final: Implementación de un algoritmo genético y programación genética

Araceli Roque Hernández¹ Inteligencia computacional-Cómputo evolutivo, 05 de mayo 2024

Instituto Nacional de Astrofísica Óptica y Electrónica, Luis Enrique Erro 1, Tonantzintla, Puebla, Mexico

Abstract. Desarrollo de un algoritmo genético para la minimización de la función de Ackley, junto con la aplicación de programación genética enfocada en resolver problemas de regresión simbólica.

1 Introducción

Los algoritmos genéticos son técnicas de optimización y aprendizaje que se inspiran en los principios de la evolución natural y la genética. Estos métodos utilizan procesos como la selección, la mutación y el cruce para resolver problemas complejos mediante la simulación de dinámicas evolutivas.

El objetivo de esta práctica es implementar un algoritmo genético diseñado específicamente para minimizar la función de Ackley, que se denota como:

$$f(\mathbf{x}) = -a \exp \left(-b \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2} \right) - \exp \left(\frac{1}{d} \sum_{i=1}^d \cos(cx_i) \right) + a + \exp(1) \quad (1)$$

donde ($a = 20$), ($b = 0.2$), ($c = 2\pi$), el intervalo de las variables es ($-32.768 \leq x_i \leq 32.768$), la solución óptima se encuentra en ($f(x^*) = 0.0$), considerando como tamaño del problema ($n = 15$).

Implementaremos dos tipos de cruce: la cruce aritmética simple y la cruce aritmética completa. Además, exploraremos el método de reemplazo generacional. El objetivo es evaluar cómo diferentes técnicas y enfoques pueden influir en los resultados. Esta comparación nos permitirá determinar qué métodos son más efectivos para optimizar nuestro algoritmo genético en distintas condiciones.

El funcionamiento del modelo generacional: Durante cada iteración se crea una población completa con nuevos individuos, es decir La nueva población reemplaza directamente a la antigua.

La programación genética(GP) es una técnica de inteligencia artificial en la que se utilizan algoritmos evolutivos para generar automáticamente programas informáticos. Basándose en los principios de selección natural y genética, una población de programas se somete a operaciones como el cruce, la mutación y la selección para evolucionar y mejorar con el tiempo. El objetivo de la GP es

generar automáticamente programas que puedan resolver un problema en concreto. Esto se logra evaluando un conjunto de programas generados previamente para determinar cuál o cuáles son los más adecuados para la resolución del problema. La selección se utiliza para crear la próxima generación de programas, que también se evaluarán y seleccionarán si es pertinente, repitiendo el proceso tantas veces como sea necesario.

Para esta practica se utilizara GP para resolver un ejercicio regresión logística. La regresión simbólica es un tipo de análisis que busca modelar datos a partir de una fórmula matemática generada automáticamente, que mejor se ajuste a dichos datos. Este método es particularmente potente porque no se limita a un conjunto predefinido de modelos, sino que puede generar y evaluar estructuras de modelo completamente nuevas y posiblemente más efectivas.

Se aplicara regresión simbólica a los datos generados por la función denotada:

$$f(x) = \sin(2x)$$

en el intervalo $[-\pi, \pi]$.

El conjunto de funciones es el siguiente:

$$F = \{+, -, *, /, \sqrt{\cdot}, \cos\}$$

se excluye la función seno.

El conjunto de terminales es el siguiente:

$$T = \{x, 1.0, 2.0, 3.0\}$$

2 Desarrollo

2.1 Minimización de la función Ackley

Para la implementación de la minimización de la función Ackley, se detallan los parámetros en la Tabla 1.

Table 1: Parámetros del algoritmo genético para minimizar la función de Ackley

Característica	Descripción
Tipo de población	Estándar
Inicialización	Aleatoria
Representación	Números reales
Tamaño de población	100
Selección	Torneo binario
Tipo de recombinación	Cruza aritmética simple con probabilidad 90%
Tipo de mutación	Uniforme con probabilidad 1/Tamaño cromosoma
Tipo de reemplazo	Estado estable, 20% hijos
Condición de paro	Umbral de error ($f(x) = 0.0001$)

Algorithm 1 Algoritmo genético para minimización de la función Ackley (Parte 1)

```

1: function FUNCIONACKLEY( $x, a, b, c$ )
2:    $n \leftarrow \text{longitud}(x)$ 
3:    $\text{suma\_cuadrados} \leftarrow \sum(x_i^2)$ 
4:    $\text{promedio\_cuadrados} \leftarrow \text{suma\_cuadrados}/n$ 
5:    $\text{termino1} \leftarrow -a \times \exp(-b \times \sqrt{\text{promedio\_cuadrados}})$ 
6:    $\text{suma\_cos} \leftarrow \sum(\cos(c \times x_i))$ 
7:    $\text{promedio\_cos} \leftarrow \text{suma\_cos}/n$ 
8:    $\text{termino2} \leftarrow -\exp(\text{promedio\_cos})$ 
9:    $\text{resultado} \leftarrow \text{termino1} + \text{termino2} + a + \exp(1)$ 
10:  return  $\text{resultado}$ 
11: end function
12: function INICIALIZACION( $\text{tamano}, \text{dimension}, \text{limite\_inferior}, \text{limite\_superior}$ )
13:  return matriz aleatoria uniforme de tamaño  $[\text{tamano}, \text{dimension}]$ 
14: end function
15: function TORNEOBINARIO( $\text{poblacion}, \text{puntuaciones}$ )
16:  for  $i \leftarrow 1$  to  $\text{longitud}(\text{poblacion})$  do
17:     $i, j \leftarrow$  índices aleatorios
18:     $\text{ganador} \leftarrow$  el mejor entre  $\text{poblacion}[i]$  y  $\text{poblacion}[j]$  basado en  $\text{puntuaciones}$ 
19:  end for
20:  return ganadores
21: end function

```

El algoritmo empleado para implementar la función Ackley, que se describe en la Tabla 1, lo podemos ver en el algoritmo 1 y 2.

Para analizar el impacto del operador de cruce en la implementación, se realizó una modificación en la función de cruce utilizando el método de *cruce aritmético completo*, estableciendo un valor de $\alpha = 0.5$, el pseudocódigo de la implementación se muestra en el algoritmo 3.

Para evaluar el impacto del método de reemplazo, se introdujo una modificación que implementa un enfoque generacional. Este método implica reemplazar íntegramente la población actual con los descendientes, facilitando así la rápida propagación de las características genéticas beneficiosas. El Pseudocódigo donde se detalla como funciona implementando el enfoque generacional lo visualizamos en el algoritmo 4.

2.2 Regresión simbólica utilizando GP

Para calcular la aptitud o el fitness de cada individuo, se utilizan 20 puntos uniformemente distribuidos de la variable independiente en el intervalo establecido. Estos puntos se emplean para evaluar la aptitud de los individuos mediante el cálculo de la suma de los errores absolutos para estos 20 puntos. El objetivo es minimizar el error; si este es igual a cero, se tendrá una expresión exacta a la que se busca. Además, el máximo número de nodos permitidos en un individuo es de

Algorithm 2 Algoritmo genético para minimización de la función Ackley (Parte 2)

```

1: function CRUCE(padre1, padre2, tasa)
2:   if número aleatorio < tasa then
3:     alpha ← número aleatorio
4:     hijo ← alpha × padre1 + (1 − alpha) × padre2
5:     return hijo
6:   else
7:     return copia de padre1
8:   end if
9: end function
10: function MUTACION(individuo, dimension, limite_inferior, limite_superior)
11:   tasa_mutacion ← 1/dimension
12:   for i ← 0 to dimension do
13:     if número aleatorio < tasa_mutacion then
14:       cambio ← número aleatorio uniforme entre − 0.1 y 0.1 ×
        (limite_superior − limite_inferior)
15:       individuo[i] ← individuo[i] + cambio
16:       individuo[i] ← limitado entre limite_inferior y limite_superior
17:     end if
18:   end for
19:   return individuo
20: end function
21: function REEMPLAZOESTADOESTABLE(poblacion, descendencia, puntuaciones,
    punt_desc, tasa_reemplazo)
22:   num_reemplazos ← tamaño de poblacion × tasa_reemplazo
23:   num_reemplazos ← min(num_reemplazos, longitud de descendencia)
24:   indices_para_reemplazar ← índices de los peores en puntuaciones
25:   nuevos_indices ← índices de los mejores en punt_desc
26:   Reemplaza en poblacion y puntuaciones según índices calculados
27:   return poblacion, puntuaciones
28: end function
29: function ALGORITMOGENETICO(funcion_objetivo, dim, limite_inf, limite_sup,
    tam_pob, gen_max, tasa_cruce)
30:   poblacion ← Inicializacion(tam_pob, dim, limite_inf, limite_sup)
31:   puntuaciones ← evalúa poblacion con funcion_objetivo
32:   for generacion ← 1 to gen_max do
33:     padres ← TorneoBinario(poblacion, puntuaciones)
34:     descendencia ← genera y muta cruzando padres
35:     punt_desc ← evalúa descendencia
36:     poblacion, puntuaciones ← ReemplazoEstadoEstable(poblacion, descendencia, puntuaciones, punt_desc, 0.2)
37:     if condición de terminación alcanzada then
38:       Imprime "Condición de terminación alcanzada"
39:       break
40:     end if
41:     if generacion%100 == 0 then
42:       Imprime estado actual
43:     end if
44:   end for
45:   return mejor solución y valor de la población
46: end function

```

Algorithm 3 Cruce aritmético completo

```

1: function CRUCEARITMÉTICOCompleto(padre1, padre2,  $\alpha = 0.5$ , tasa = 0.9)
2:   if rand() < tasa then
3:      $hijo \leftarrow \alpha \times padre1 + (1 - \alpha) \times padre2$ 
4:     return hijo
5:   end if
6:   return copy(padre1)
7: end function

```

Algorithm 4 Algoritmo genético con modelo generacional

Require: *funcion_objetivo*, *dim*, *limite_inferior*, *limite_superior*, *tam_poblacion*, *gen_max*, *tasa_cruce*

```

1: Inicializar población con tam_poblacion individuos de dimensiones dim, valores
   entre limite_inferior y limite_superior
2: Evaluar cada individuo en la población usando funcion_objetivo
3: for cada generación hasta gen_max do
4:   Seleccionar padres mediante torneo binario
5:   Cruzar padres para formar descendientes con probabilidad tasa_cruce
6:   Mutar descendientes con una probabilidad determinada
7:   Evaluar cada descendiente usando funcion_objetivo
8:   Reemplazar completamente la población actual con los descendientes
9:   Imprimir la mejor puntuación de la población actual
10:  if se cumple la condición de terminación then
11:    Terminar el ciclo
12:  end if
13: end for
14: return el mejor individuo y su valor de aptitud

```

25. El pseudocódigo podemos observarlo en el algoritmo 5, con los parámetros descritos en la tabla 2.

Table 2: Parámetros para GP, de la regresión simbólica

Característica	Descripción
Tipo de población	Estándar
Inicialización	Ramped half and half
Representación	Árbol
Tamaño de población	500
Selección	Torneo binario
Probabilidad recombinación/mutación	90% / 1%
Tipo de recombinación	0.9%
Tipo de mutación	0.25 %
Tipo de reemplazo	generacional con etilismo, 9%
Condición de paro	Mejor solución del umbral ($f(x) = 0.1$)

Algorithm 5 Algoritmo genético para regresión simbólica

Require: *funcion_objetivo*, *dim*, *limite_inferior*, *limite_superior*, *tam_poblacion*, *gen_max*, *tasa_cruce*

- 1: Inicializar población con *tam_poblacion* individuos de dimensiones *dim*, valores entre *limite_inferior* y *limite_superior*
- 2: Evaluar cada individuo en la población usando *funcion_objetivo*
- 3: **for** cada generación hasta *gen_max* **do**
- 4: Seleccionar padres mediante torneo binario
- 5: Cruzar padres para formar descendientes con probabilidad *tasa_cruce*
- 6: Mutar descendientes con una probabilidad determinada
- 7: Evaluar cada descendiente usando *funcion_objetivo*
- 8: Reemplazar completamente la población actual con los descendientes
- 9: **if** se cumple la condición de terminación **then**
- 10: Terminar el ciclo
- 11: **end if**
- 12: **end for**
- 13: **return** el mejor individuo y su valor de aptitud

3 Resultados

Los resultados de la implementación de los dos algoritmos se muestran a continuación.

3.1 Resultados para la minimización de la función Ackley

Considerando la tabla de especificación en la tabla 1 del algoritmo, se realizaron 10 ejecuciones, con una generación máxima de 3000 y 10000 generaciones, obteniendo los resultados mostrados en la tabla...

Table 3: Resultados para la generación de 3000 y 10000, utilizando los parametros de la tabla 1

Ejecución	GEN_MAX = 3000	GEN_MAX = 10000
1	0.0031107257360649676	0.00046584011941464
2	0.0012761035657544717	0.00046223915248022607
3	0.0028794887450094286	0.0031107257360649676
4	0.0038297517215686305	0.00020732355247732315
5	0.0050206406808794846	0.0008853865293301233
6	0.0021846559939970867	0.0008123519941034552
7	0.0016207348538128663	0.0007305985537340298
8	0.0022114750042345754	0.0006755608500594867
9	0.0027877410639836064	0.0003614738801371864
10	0.004864414218197499	0.000444306635298819
Promedio	0.0029785731583502617	0.0008155807003100257
Desv. Estándar	0.0012031422087993389	0.0007912398153452062

Tras analizar los datos obtenidos en la tabla 2, incluyendo el promedio y la desviación estándar, observamos que incrementar el número de generaciones de 3000 a 10000 mejora notablemente los resultados. Los valores resultantes son menores y se aproximan más al objetivo de 0.0.

Específicamente, para un máximo de 3000 generaciones tras 10 ejecuciones, el promedio fue de aproximadamente 0.00298, con una desviación estándar de 0.00120. En contraste, para un máximo de 10000 generaciones después de 10 ejecuciones, el promedio se redujo a aproximadamente 0.000816 y la desviación estándar a 0.000791.

Estos resultados nos indican que incrementar el número de generaciones conduce a una optimización más efectiva. Al hacerlo, se logran soluciones de mayor calidad y una mayor estabilidad en los resultados. Aunque el algoritmo no convergió hacia la solución óptima, al alcanzar las 10,000 generaciones se obtuvo un promedio favorable de 0.0008.

Después de modificar el operador de cruza y realizar 10 ejecuciones se obtuvieron los resultados mostrados en la tabla 3.

Con los resultados obtenidos en la tabla 3 podemos notar que de igual forma en la generación máxima de 10000, son consistentes más bajos que para una generación máxima de 3000.

El promedio de los resultados para la generación de 10000 es considerablemente menor (0.000576) que el promedio para la generación 3000 (0.003219). Esto sugiere que aumentar el número de generaciones permite al algoritmo refinar

Table 4: Resultados de ejecuciones con una generación de 3000 y 10000, utilizando el operador de cruce aritmético completo

Ejecución	GEN_MAX = 3000	GEN_MAX = 10000
1	0.004856083247876786	0.0010201453395599636
2	0.0037547844430503297	0.00024187932151820135
3	0.0030049592937335667	0.00028696782442994007
4	0.0025923424275089424	0.0011153751067571882
5	0.004966066091683263	0.0008134276086413372
6	0.0016968411427957797	0.0003723997305056237
7	0.002757536313609865	0.0007625026829782833
8	0.0026046351413460123	0.00012423210089496095
9	0.003382001752225694	0.0008509622431032149
10	0.002573646663361462	0.00017424920583097858
Promedio	0.00321888965171917	0.0005762141164219692
Desviación Estándar	0.0009903671275968869	0.0003545527769653818

sus soluciones y obtener resultados más optimizados en promedio. Con respecto a la desviación estándar, que mide la variabilidad de los resultados, es significativamente menor en las ejecuciones con 10000 generaciones (0.000355) comparado con las de 3000 generaciones (0.000990). Esta reducción en la desviación estándar refleja una mayor consistencia y fiabilidad en los resultados cuando el algoritmo tiene más generaciones para trabajar.

Considerando los resultados de dos configuraciones distintas de cruce en la minimización de la función Ackley, observamos diferencias significativas. La Tabla 2 implementó un cruce aritmético simple con una probabilidad del 90%, mientras que la Tabla 3 aplicó un cruce aritmético completo con un $\alpha = 0.5$.

Al comparar ambos métodos para la generación de 10000, el cruce aritmético completo demostró ser más consistente y eficaz en la minimización de la función objetivo que el cruce aritmético simple. Esta eficacia se refleja en una mayor consistencia en los resultados y en la obtención de valores mínimos más bajos, indicando una exploración más efectiva del espacio de soluciones.

- **Mejor consistencia con cruce aritmético completo:** Este método no solo proporciona una mayor uniformidad en los resultados, sino que también logra minimizar mejor la función objetivo, lo cual sugiere una mayor eficiencia en la exploración y explotación del espacio de soluciones.
- **Importancia del número de generaciones:** Incrementar el número de generaciones resulta en una mejora continua de los resultados. Este aumento permite al algoritmo optimizar más efectivamente sus soluciones y alcanzar niveles más profundos de precisión.

Los resultados al modificar el criterio de reemplazo por uno generacional se muestran en la tabla 4.

Table 5: Resultados para la generación 3000 y 10000 utilizando un enfoque generacional

Número	GEN_MAX = 3000	GEN_MAX = 10000
1	2.646361612163663	2.8600216612659435
2	2.554066721149184	2.233550984981076
3	2.9862079027266373	2.8166823679267208
4	2.1895349579177865	2.223671752298341
5	2.599266445389692	2.876906867400376
6	3.2378042456232063	2.4924573493836104
7	2.749795602541852	2.391328620519801
8	2.654090717655905	2.2059605407642207
9	3.2563893184843704	3.0154139068842842
10	3.0568842772310743	2.749706804450572
Promedio	2.79304018	2.5865700855874945
Desviación Estándar	0.31953118	0.29507647039519264

3.2 Resultados para la la regresión simbólica

EJEC.	GEM	PROMEDIO	DESV.EST.	MEJOR FITNESS
1	48	0.14220892049680478	0.01303253192321890	0.09764166518384748
2	13	0.38463111764537744	0.07345455095692106	0.0694563703359794
4	74	0.11083007980064494	0.003250170326438248	0.09867522404316216
5	358	0.10353671495971338	0.0002662268837457821	0.09774497931343981
6	108	0.1026542169311231	0.0004561645672305038	0.09915259136582533

Table 6: Resultados de las ejecuciones del algoritmo para la regresión simbólica

Como podemos observar en la tabla 5, El número de generaciones varía significativamente entre las ejecuciones (desde 13 hasta 358). Esto nos indica que existen diferencias en la convergencia del algoritmo bajo diferentes condiciones. Una menor cantidad de generaciones podrían sugerirnos una rápida convergencia pero no siempre nos indica que es un mejor resultado global. La relación entre desviación estándar y convergencia: Hay una tendencia de disminución en la desviación estándar a medida que aumenta el número de generaciones. La ejecución con el mejor fitness es la número 4, con un valor de 0.09867522404316216. Esta ejecución también tiene una desviación estándar bastante baja, lo que sugiere que la población de soluciones estaba bastante concentrada alrededor del promedio, indicando menor variabilidad entre los individuos. Aunque la ejecución 4 no tuvo el mayor número de generaciones, logró el mejor fitness, lo cual sugiere que el algoritmo fue eficiente en encontrar una buena solución en un número moderado de generaciones.

4 Conclusiones

A partir de las tablas y resultados presentados sobre la implementación del algoritmo genético para minimizar la función Ackley, podemos llegar a las siguientes conclusiones:

- **Impacto de aumentar las generaciones:** El aumento en el número de generaciones mejora la consistencia de los resultados. Este incremento permite explorar más eficazmente el espacio de soluciones, mejorando así la capacidad del algoritmo para encontrar soluciones óptimas.
- **Comparación de métodos de cruce:** Entre el cruce aritmético simple y el cruce aritmético completo, observamos que el método completo fue más efectivo. Este mostró mayor consistencia y alcanzó valores más bajos, demostrando ser más robusto en distintas ejecuciones.
- **Evaluación del Enfoque Generacional:** El enfoque generacional, que implica reemplazar completamente la población mientras se mantiene el mejor individuo o los mejores de cada generación, no resultó tan efectivo como esperado, dado que los valores obtenidos se alejaron más del ideal de 0.0. Un enfoque generacional con elitismo podría haber sido más beneficioso, asegurando la retención de las características genéticas deseables mientras se renueva la población.

Las variaciones en los operadores de cruce destacan la importancia de seleccionar un método que complemente la naturaleza del problema y la configuración del algoritmo. Por ejemplo, el cruce aritmético completo con un parámetro $\alpha = 0.5$ proporcionó una búsqueda más eficaz y eficiente en el espacio de soluciones comparado con un cruce aritmético simple con una alta probabilidad de cruce.

Estos resultados subrayan la importancia de una cuidadosa selección de parámetros en algoritmos genéticos, como el número de generaciones, el tipo de cruce y la estrategia de reemplazo. Ajustar estos parámetros puede mejorar significativamente el rendimiento del algoritmo en términos de precisión y estabilidad, aunque en este caos no logramos que el algoritmo convergiera a 0.0, tuvimos un promedio aproximado de 0.000444306635298819 utilizando los parámetros iniciales proporcionados en la tabla 1. esos fueron nuestros mejores resultados.

La programación genética utilizado para la regresión simbólica muestra una notable variabilidad en su rendimiento, influenciada fuertemente por las configuraciones iniciales y las condiciones específicas de cada ejecución. Esta variabilidad se manifiesta en cómo las diferentes configuraciones del tamaño de la población, tasas de mutación y cruce, y métodos de selección afectan tanto la velocidad de convergencia como la calidad de las soluciones obtenidas. La diversidad genética dentro de la población es crucial, ya que una baja diversidad puede llevar a una convergencia prematura hacia óptimos locales, mientras que una alta diversidad favorece una exploración más amplia pero puede ralentizar la convergencia.

La eficacia de los operadores genéticos, como la mutación y el cruce, es determinante en la generación de nuevas soluciones y en la introducción de vari-

abilidad, impactando directamente la capacidad del algoritmo para mejorar continuamente las soluciones a través de generaciones.

En resumen, la adaptación y el ajuste meticuloso del algoritmo genético a las características específicas del problema y las condiciones iniciales son clave para mejorar su eficacia y asegurar que se puedan alcanzar soluciones de alta calidad de manera consistente.

References

1. Eiben, A. E., Smith, J. E. (2015). Natural Computing Series Introduction to Evolutionary Computing. In Natural Computing Series. www.springer.com/series/